1. Setup e Imports

```
import json as _json
import json
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm import tqdm
import os
import sys
from pathlib import Path
import subprocess

# Agregar path del proyecto para imports
sys.path.append(os.path.abspath('../../..'))
from sae.tools.naming_utils import save_checkpoint, get_model_name, get_extras_id, get_checkp
from sae.tools.experiment_utils import get_experiment_dir, get_plots_dir, get_metrics_dir, ge

# Configuración de visualización para Quarto/PDF
pd.set_option('display.width', 100)
pd.set_option('display.max_columns', None)
np.set_printoptions(linewidth=100, precision=4, suppress=True)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Dispositivo: {device}')
```

```
Dispositivo: cuda
```

2 Metadata para Naming de Checkpoints

```
# Metadata para naming de checkpoints (separada de hiperparámetros)
NAMING_META = {
    'variante': 'matryoshkabatchtopk',
    'tecnica':  'matryoshkabatchtopk',
    'capa':     6,
    'juegos':   12500,
    'base_path': 'C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt',
    'experiment_base_path': os.path.abspath('../../../experiments'),
    'extras': {
        'expansion':         16,
        'k':                 50,
        'matryoshka_widths': [2048, 4096, 8192],
        'epochs':            1000,
        'patience':          20
    }
}

print('\nMetadata de naming:')
for key, value in NAMING_META.items():
    print(f'  {key}: {value}')
```

```
Metadata de naming:
  variante: matryoshkabatchtopk
  tecnica: matryoshkabatchtopk
  capa: 6
  juegos: 12500
  base_path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt
  experiment_base_path: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments
  extras: {'expansion': 16, 'k': 50, 'matryoshka_widths': [2048, 4096, 8192], 'epochs': 1000
```

```
extras_id = get_extras_id(
    get_checkpoint_dir(
        NAMING_META['base_path'],
        NAMING_META['variante'],
        NAMING_META['tecnica'],
        NAMING_META['capa'],
        NAMING_META['juegos']
    ),
```

```
    NAMING_META['extras']
) if NAMING_META.get('extras') else None

# get_plots_dir crea el directorio automáticamente
plots_dir = Path(get_plots_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
))

print(f'extras_id: {extras_id}')
print(f'plots_dir: {plots_dir}')
```

```
extras_id: ex2
plots_dir: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_matryos
```

3. Configuración del SAE

Arquitectura MatryoshkaBatchTopK: - **Nested reconstruction losses**: se calculan pérdidas MSE en múltiples sub-anchos (`matryoshka_widths`) — cada ancho debe reconstruir el input por sí solo - **BatchTopK activation**: threshold global del batch para que el promedio de features activas sea `k` - **Sin coeficiente de sparsidad**: la sparsity se controla directamente con `k` - **Loss = Σ MSE(recon_w, x)** para cada ancho `w` `matryoshka_widths`

Parámetro	L1 (standard)	BatchTopK	Matryoshka-BatchTopK
Sparsity	`sparsity_coef` (soft)	`k` (promedio del batch)	`k` (promedio, por nivel)
L0	Variable	Variable por muestra	Variable por muestra, por nivel
Loss	MSE + · L1	MSE	Σ MSE_width

Las celdas siguientes entrenan el SAE directamente sobre los archivos `.npy` pre-extraídos usando la arquitectura MatryoshkaBatchTopK.

```
NUM_EPOCHS = NAMING_META['extras']['epochs']

cfg = {
    'architecture':            'matryoshkabatchtopk',
    'd_in':                    512,
    'd_sae':                   8192,
    'k':                       50,
    'matryoshka_widths':       [2048, 4096, 8192],
    'lr':                      3e-4,
    'train_batch_size_tokens': 4096,
    'hook_name':               'blocks.5.hook_resid_post',
    # P3: betas según config.py de los autores (beta2=0.99 vs default PyTorch 0.999)
    'beta1':                   0.9,
    'beta2':                   0.99,
    # P1: parámetros para auxiliary loss de dead features (config.py de los autores)
    'n_batches_to_dead':       20,    # batches sin activación para declarar feature muerta
    'top_k_aux':               512,   # top-k features muertas usadas en L_aux
    'aux_penalty':             1/32,  # peso  de L_aux (paper ec. 5)
    # P2: normalización del input por muestra
    'input_unit_norm':         True,
}

cfg_summary = pd.DataFrame({
    'Parámetro': ['architecture', 'd_in', 'd_sae', 'k (L0 promedio)', 'matryoshka_widths',
                  'hook_name', 'lr', 'beta1', 'beta2',
                  'n_batches_to_dead', 'top_k_aux', 'aux_penalty', 'input_unit_norm'],
    'Valor': [cfg['architecture'], cfg['d_in'], cfg['d_sae'], cfg['k'],
              str(cfg['matryoshka_widths']), cfg['hook_name'], cfg['lr'],
              cfg['beta1'], cfg['beta2'],
              cfg['n_batches_to_dead'], cfg['top_k_aux'], cfg['aux_penalty'],
              cfg['input_unit_norm']]
})
print('Configuración SAELens:')
print(cfg_summary.to_string(index=False))
```

```
Configuración SAELens:
        Parámetro                  Valor
     architecture matryoshkabatchtopk
             d_in                    512
            d_sae                   8192
  k (L0 promedio)                     50
matryoshka_widths  [2048, 4096, 8192]
```

```
        hook_name blocks.5.hook_resid_post
               lr                   0.0003
            beta1                      0.9
            beta2                     0.99
n_batches_to_dead                       20
        top_k_aux                      512
      aux_penalty                  0.03125
  input_unit_norm                     True
```

4. Dataset de Activaciones (Train/Validation Split 80-20%)

Activaciones pre-extraídas con `sae/activations/run_extraction.py` desde la capa `blocks.5.hook_resid_post`.

```
class ActivationsDataset(Dataset):
    """Dataset para cargar activaciones extraídas"""
    def __init__(self, activations_path):
        self.activations = np.load(activations_path, mmap_mode='r')
    def __len__(self):
        return len(self.activations)
    def __getitem__(self, idx):
        return torch.tensor(self.activations[idx], dtype=torch.float32)

activations_file = Path(f"../../../activations/data/layer{NAMING_META['capa']}_{NAMING_META[
full_dataset = ActivationsDataset(activations_file)

# Split 80-20 reproducible
train_size = int(0.8 * len(full_dataset))
val_size   = len(full_dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(
    full_dataset, [train_size, val_size],
    generator=torch.Generator().manual_seed(42)
)

train_loader = DataLoader(train_dataset, batch_size=cfg["train_batch_size_tokens"], shuffle=
val_loader   = DataLoader(val_dataset,   batch_size=cfg["train_batch_size_tokens"], shuffle=

dataset_info = pd.DataFrame({
    'Conjunto': ['Train', 'Validación', 'Total'],
    'Muestras': [f'{len(train_dataset):,}', f'{len(val_dataset):,}', f'{len(full_dataset):,}
    'Shape':    [str(full_dataset.activations.shape), '', ''],
```

```
})
print(dataset_info.to_string(index=False))
```

```
  Conjunto Muestras         Shape
     Train  590,000 (737500, 512)
Validación  147,500
     Total  737,500
```

5. Modelo: MatryoshkaBatchTopK SAE

Implementación de la arquitectura MatryoshkaBatchTopK según el paper (ec. 3-5) y el código de los autores (`sae.py`).

Arquitectura: - El encoder proyecta **una sola vez** con `W_enc` completo (`d_in → d_sae`) para obtener `f(x)` de tamaño `d_sae` (paper ec. 3) - **BatchTopK global único**: se seleccionan los `k × batch_size` valores más altos sobre los `d_sae × batch_size` valores del batch completo — un único threshold compartido entre todos los anchos (paper ec. 6) - Las reconstrucciones por nivel usan **prefijos del mismo vector**: `all_acts_topk[:, 0:w]` — las features `0:2048` son literalmente las mismas en el nivel `w=2048` y en el nivel `w=4096` (paper ec. 4) - En **eval**, se usa un threshold persistente calibrado con EMA durante training — no se recalcula (paper ec. 7) - **L0 es variable por muestra pero promedia k** en cada batch

Parámetro	L1 (standard)	BatchTopK	Matryoshka-BatchTopK
Sparsity	`sparsity_coef` (soft)	`k` (promedio del batch)	`k` (promedio, threshold global)
L0	Variable	Variable por muestra	Variable por muestra
Loss	MSE + · L1	MSE	Σ MSE_width + · L_aux

```
class MatryoshkaBatchTopKSAE(nn.Module):
    """
    Sparse Autoencoder con activación MatryoshkaBatchTopK.
    Corregido según paper (ec. 3-5) e implementación de los autores (sae.py).

    Correcciones aplicadas:
    - P1: encoding único con W_enc completo; BatchTopK global; prefijos compartidos
    - P1: BatchTopK preserva valores originales (x * indicator), threshold EMA para eval
```

```
    - P1: auxiliary loss para dead features (sae.py:198-221)
    - P2: make_decoder_weights_and_grad_unit_norm proyecta gradiente (sae.py:51-57)
    - P2: preprocessing input_unit_norm configurable (sae.py:36-43)
    - P3: W_dec inicializado como transpuesta de W_enc (sae.py:93-94)
    """
    def __init__(self, d_in, d_sae, k, matryoshka_widths,
                 n_batches_to_dead=20, top_k_aux=512, aux_penalty=1/32,
                 input_unit_norm=False):
        super().__init__()
        self.d_in  = d_in
        self.d_sae = d_sae
        self.k     = k
        self.matryoshka_widths = sorted(matryoshka_widths)
        assert self.matryoshka_widths[-1] == d_sae, \
            f"El último ancho ({self.matryoshka_widths[-1]}) debe ser d_sae ({d_sae})"

        self.n_batches_to_dead = n_batches_to_dead
        self.top_k_aux         = top_k_aux
        self.aux_penalty       = aux_penalty
        self.input_unit_norm   = input_unit_norm

        self.W_enc = nn.Parameter(torch.empty(d_in, d_sae))
        self.W_dec = nn.Parameter(torch.empty(d_sae, d_in))
        self.b_dec = nn.Parameter(torch.zeros(d_in))

        # P3: W_dec = W_enc^T normalizado (sae.py:93-94) - inicialización tied
        nn.init.kaiming_uniform_(self.W_enc)
        self.W_dec.data[:] = self.W_enc.t().data
        self.W_dec.data[:] = self.W_dec / self.W_dec.norm(dim=-1, keepdim=True)

        # P1: threshold persistente calibrado con EMA durante training (sae.py:97)
        self.register_buffer('threshold', torch.tensor(0.0))
        # P1: contador de batches sin activación por feature - para L_aux (sae.py:30-32)
        self.register_buffer('num_batches_not_active', torch.zeros(d_sae))

    # ------------------------------------------------------------------
    # P2: Preprocessing del input (sae.py:36-48)
    # ------------------------------------------------------------------

    def preprocess_input(self, x):
        if self.input_unit_norm:
            x_mean = x.mean(dim=-1, keepdim=True)
```

```
            x = x - x_mean
            x_std = x.std(dim=-1, keepdim=True)
            x = x / (x_std + 1e-5)
            return x, x_mean, x_std
        return x, None, None

    def postprocess_output(self, x_reconstruct, x_mean, x_std):
        if self.input_unit_norm and x_mean is not None:
            x_reconstruct = x_reconstruct * x_std + x_mean
        return x_reconstruct

    # ------------------------------------------------------------------
    # P1: Encoding único + BatchTopK global (sae.py:100-119)
    # ------------------------------------------------------------------

    def compute_activations(self, x_cent):
        # P1: encoding ÚNICO con W_enc completo - un solo pase para todos los anchos
        # Las features 0:w son literalmente las mismas en todos los niveles (paper ec. 3)
        pre_acts = x_cent @ self.W_enc     # [batch, d_sae]
        acts     = F.relu(pre_acts)         # ReLU antes del topk (sae.py:102)

        if self.training:
            # P1: BatchTopK global sobre d_sae×batch_size valores (paper ec. 6)
            # Preserva valores originales con scatter - NO resta threshold (sae.py:104-114)
            topk_result = torch.topk(acts.flatten(), self.k * x_cent.shape[0])
            acts_topk = (
                torch.zeros_like(acts.flatten())
                .scatter(-1, topk_result.indices, topk_result.values)
                .reshape(acts.shape)
            )
            self._update_threshold(acts_topk)
        else:
            # P1: eval usa threshold persistente, no recalcula (paper ec. 7, sae.py:117)
            acts_topk = torch.where(acts > self.threshold, acts, torch.zeros_like(acts))

        return acts, acts_topk

    @torch.no_grad()
    def _update_threshold(self, acts_topk, lr=0.01):
        # P1: EMA del mínimo valor positivo - calibra  para eval (sae.py:224-228)
        positive_mask = acts_topk > 0
        if positive_mask.any():
```

```
            min_positive = acts_topk[positive_mask].min()
            self.threshold = (1 - lr) * self.threshold + lr * min_positive

    @torch.no_grad()
    def _update_inactive_features(self, acts_topk):
        # P1: necesario para L_aux - trackea qué features no se activan (sae.py:60-62)
        self.num_batches_not_active += (acts_topk.sum(0) == 0).float()
        self.num_batches_not_active[acts_topk.sum(0) > 0] = 0

    # ---------------------------------------------------------------
    # Forward
    # ---------------------------------------------------------------

    def forward(self, x):
        """
        Retorna:
            outputs      : lista de (recon_w, hidden_w) para cada ancho matryoshka
                           recon_w está en espacio normalizado (= espacio original si input_u
            all_acts     : pre-topk [batch, d_sae] - para L_aux
            all_acts_topk: post-topk [batch, d_sae]
            x_proc       : x preprocesado (= x si input_unit_norm=False)
        """
        x_proc, x_mean, x_std = self.preprocess_input(x)
        x_cent = x_proc - self.b_dec
        all_acts, all_acts_topk = self.compute_activations(x_cent)

        # P1: reconstrucciones por PREFIJO compartido (paper ec. 4, sae.py:149-155)
        # all_acts_topk[:, 0:w] son las mismas features en todos los niveles anidados
        # recon_w se mantiene en espacio normalizado para que la loss compare espacios iguale
        outputs = []
        for w in self.matryoshka_widths:
            hidden_w = all_acts_topk[:, :w]
            recon_w  = hidden_w @ self.W_dec[:w, :] + self.b_dec
            outputs.append((recon_w, hidden_w))

        self._update_inactive_features(all_acts_topk)
        return outputs, all_acts, all_acts_topk, x_proc

    # ---------------------------------------------------------------
    # P1: Auxiliary loss para dead features (sae.py:198-221)
    # ---------------------------------------------------------------
```

```
    def get_auxiliary_loss(self, x_proc, x_reconstruct_full, all_acts):
        residual        = x_proc.float() - x_reconstruct_full.float()
        aux_reconstruct = torch.zeros_like(residual)

        dead_features = self.num_batches_not_active >= self.n_batches_to_dead

        if dead_features.sum() > 0:
            acts_topk_aux = torch.topk(
                all_acts[:, dead_features],
                min(self.top_k_aux, int(dead_features.sum())),
                dim=-1,
            )
            acts_aux = torch.zeros_like(all_acts[:, dead_features]).scatter(
                -1, acts_topk_aux.indices, acts_topk_aux.values
            )
            aux_reconstruct = aux_reconstruct + acts_aux @ self.W_dec[dead_features]

        if aux_reconstruct.abs().sum() > 0:
            return self.aux_penalty * (aux_reconstruct.float() - residual.float()).pow(2).mea
        return torch.tensor(0.0, device=x_proc.device)

    # --------------------------------------------------------------------
    # P2: Constraint de norma unitaria con proyección del gradiente (sae.py:51-57)
    # --------------------------------------------------------------------

    @torch.no_grad()
    def make_decoder_weights_and_grad_unit_norm(self):
        # P2: proyecta gradiente al espacio tangente de la esfera unitaria + normaliza
        # @torch.no_grad() evita trazar grafo interno - no impide leer .grad (sae.py:50)
        # Llamar ANTES de optimizer.step() (training.py:31)
        W_dec_normed    = self.W_dec / self.W_dec.norm(dim=-1, keepdim=True)
        W_dec_grad_proj = (
            (self.W_dec.grad * W_dec_normed).sum(-1, keepdim=True) * W_dec_normed
        )
        self.W_dec.grad -= W_dec_grad_proj
        self.W_dec.data  = W_dec_normed


# Crear modelo
model = MatryoshkaBatchTopKSAE(
    d_in=cfg['d_in'],
    d_sae=cfg['d_sae'],
```

```
    k=cfg['k'],
    matryoshka_widths=cfg['matryoshka_widths'],
    n_batches_to_dead=cfg['n_batches_to_dead'],
    top_k_aux=cfg['top_k_aux'],
    aux_penalty=cfg['aux_penalty'],
    input_unit_norm=cfg['input_unit_norm'],
).to(device)
num_params = sum(p.numel() for p in model.parameters())

model_info = pd.DataFrame({
    'Componente': [
        'Input', 'Encoder (W_enc)', 'Activación', 'Anchos matryoshka',
        'Hidden activos (por nivel)', 'Decoder (W_dec)', 'Output', 'Total params'
    ],
    'Dimensión': [
        f'{cfg["d_in"]}',
        f'{cfg["d_in"]} -> {cfg["d_sae"]} (único pase, prefijos compartidos)',
        f'BatchTopK global (k={cfg["k"]}) - threshold EMA en eval',
        str(cfg['matryoshka_widths']),
        f' {cfg["k"]} activos (promedio del batch)',
        f'{cfg["d_sae"]} -> {cfg["d_in"]} (slices de prefijo)',
        f'{cfg["d_in"]}',
        f'{num_params:,}'
    ]
})
print(model_info.to_string(index=False))
```

```
               Componente                                        Dimensión
                    Input                                              512
          Encoder (W_enc)  512 -> 8192 (único pase, prefijos compartidos)
               Activación BatchTopK global (k=50) - threshold EMA en eval
         Anchos matryoshka                                  [2048, 4096, 8192]
Hidden activos (por nivel)               50 activos (promedio del batch)
          Decoder (W_dec)                8192 -> 512 (slices de prefijo)
                   Output                                              512
             Total params                                        8,389,120
```

6. Función de Pérdida MatryoshkaBatchTopK

Loss = mean MSE(recon_w, x) para cada ancho w matryoshka_widths + · L_aux

- **MSE en cada nivel**: error de reconstrucción desde cada sub-espacio matryoshka
- **Sin L1 penalty**: la sparsity está controlada por el threshold global de BatchTopK
- **Loss total = media de MSE en todos los anchos** — mantiene la escala estable y preserva el balance entre reconstrucción y aux_loss independientemente del número de anchos (alineado con Bussmann `sae.py:172`)
- La loss anidada fuerza a que los sub-espacios más pequeños también sean representaciones completas del input

```
def loss_function(x_proc, outputs, all_acts, model):
    """
    Pérdida MatryoshkaBatchTopK: media de MSE en cada ancho + auxiliary loss.

    Paper ec. 5: L(x) = Σ_{m M} ‖x - x̂‖² +  ·L_aux
    Bussmann (sae.py): promedia dividiendo por n_grupos - mantiene la escala de
    la MSE estable independientemente del número de anchos, y preserva el balance
    entre reconstrucción y aux_loss.

    Args:
        x_proc:  activaciones preprocesadas [batch, d_in] (= x si input_unit_norm=False)
        outputs: lista de (recon_w, hidden_w) por ancho matryoshka
        all_acts: activaciones pre-topk [batch, d_sae] - para L_aux
        model:   referencia al modelo (para get_auxiliary_loss)
    Returns:
        total_loss, mse_per_width, aux_loss
    """
    x_reconstruct_full = outputs[-1][0]   # reconstrucción del ancho máximo

    # Media de MSE en todos los anchos + término b_dec (alineado con Bussmann sae.py:163-172)
    mse_bdec      = F.mse_loss(model.b_dec.expand_as(x_proc), x_proc)
    mse_per_width = [F.mse_loss(recon_w, x_proc) for recon_w, _ in outputs]
    mean_mse      = (mse_bdec + sum(mse_per_width)) / (len(mse_per_width) + 1)

    # P1: auxiliary loss para dead features (paper ec. 5:  ·L_aux, sae.py:198-221)
    aux_loss = model.get_auxiliary_loss(x_proc, x_reconstruct_full, all_acts)

    return mean_mse + aux_loss, mse_per_width, aux_loss


# P3: betas según config.py de los autores - beta2=0.99 vs default PyTorch 0.999
# (config.py: beta1=0.9, beta2=0.99)
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=cfg["lr"],
```

```
    betas=(cfg["beta1"], cfg["beta2"])
)

optim_info = pd.DataFrame({
    'Parámetro': ['Optimizador', 'lr', 'beta1', 'beta2',
                  'k (L0 promedio)', 'Anchos matryoshka',
                  'aux_penalty', 'n_batches_to_dead', 'top_k_aux'],
    'Valor':     ['Adam', cfg["lr"], cfg["beta1"], cfg["beta2"],
                  cfg["k"], str(cfg['matryoshka_widths']),
                  cfg['aux_penalty'], cfg['n_batches_to_dead'], cfg['top_k_aux']]
})
print(optim_info.to_string(index=False))
```

```
        Parámetro              Valor
      Optimizador               Adam
               lr             0.0003
            beta1                0.9
            beta2               0.99
  k (L0 promedio)                 50
Anchos matryoshka [2048, 4096, 8192]
      aux_penalty            0.03125
n_batches_to_dead                 20
        top_k_aux                512
```

7. Loop de Entrenamiento

```
PATIENCE = NAMING_META['extras']['patience']
WIDTHS   = cfg['matryoshka_widths']

save_dir = Path('./saved_model')
save_dir.mkdir(exist_ok=True)
best_model_name = get_model_name(
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='best',
    extras_id=extras_id
)
BEST_CKPT = save_dir / best_model_name
```

```
history = {
    'train_total_loss':      [],
    'train_aux_loss':        [],
    'train_l0_sparsity':     [],
    'train_fraction_active': [],
    'val_total_loss':        [],
    'val_aux_loss':          [],
    'val_l0_sparsity':       [],
    'val_fraction_active':   [],
}
for w in WIDTHS:
    history[f'val_mse_w{w}'] = []

best_val_loss     = float('inf')
best_epoch        = 0
epochs_no_improve = 0

print(f'Iniciando entrenamiento: {NUM_EPOCHS} épocas máximo')
print(f'Early stopping: patience={PATIENCE}')
print(f'k = {cfg["k"]} features activas (L0 promedio  k por batch)')
print(f'Anchos matryoshka: {WIDTHS}')
print(f'Best checkpoint: {BEST_CKPT}')
print('=' * 70)

for epoch in range(NUM_EPOCHS):

    # ------------------------------------------------------------------ TRAIN
    model.train()
    t_total = t_aux = t_l0 = t_frac = 0.0

    for batch in train_loader:
        x = batch.to(device)
        optimizer.zero_grad()

        # P1: forward devuelve (outputs, all_acts, all_acts_topk, x_proc)
        outputs, all_acts, all_acts_topk, x_proc = model(x)
        total_loss, _, aux_loss_val = loss_function(x_proc, outputs, all_acts, model)
        total_loss.backward()

        # P2: proyectar gradiente al espacio tangente de la esfera + normalizar
        # ANTES de optimizer.step() (sae.py:51-57, training.py:31)
        model.make_decoder_weights_and_grad_unit_norm()
```

```
        optimizer.step()
        # REMOVED: model.normalize_decoder() - reemplazado por make_decoder_weights_and_grad

        with torch.no_grad():
            hidden_full = outputs[-1][1]
            t_total += total_loss.item()
            t_aux   += aux_loss_val.item()
            t_l0    += (hidden_full > 0).float().sum(dim=1).mean().item()
            t_frac  += (hidden_full > 0).float().mean().item()

    n = len(train_loader)
    history['train_total_loss'].append(t_total / n)
    history['train_aux_loss'].append(t_aux / n)
    history['train_l0_sparsity'].append(t_l0 / n)
    history['train_fraction_active'].append(t_frac / n)

    # ------------------------------------------------------------------ VALIDATION
    model.eval()
    v_total = v_aux = v_l0 = v_frac = 0.0
    v_per_width = {w: 0.0 for w in WIDTHS}

    with torch.no_grad():
        for batch in val_loader:
            x = batch.to(device)
            outputs, all_acts, all_acts_topk, x_proc = model(x)
            total_loss, mse_per_width, aux_loss_val = loss_function(x_proc, outputs, all_acts
            hidden_full = outputs[-1][1]

            v_total += total_loss.item()
            v_aux   += aux_loss_val.item()
            v_l0    += (hidden_full > 0).float().sum(dim=1).mean().item()
            v_frac  += (hidden_full > 0).float().mean().item()
            for w, mse_w in zip(WIDTHS, mse_per_width):
                v_per_width[w] += mse_w.item()

    nv = len(val_loader)
    history['val_total_loss'].append(v_total / nv)
    history['val_aux_loss'].append(v_aux / nv)
    history['val_l0_sparsity'].append(v_l0 / nv)
    history['val_fraction_active'].append(v_frac / nv)
    for w in WIDTHS:
        history[f'val_mse_w{w}'].append(v_per_width[w] / nv)
```

```
    width_str = ' | '.join(f'w{w}={history[f"val_mse_w{w}"][-1]:.4f}' for w in WIDTHS)
    print(f"Época {epoch+1:3d}/{NUM_EPOCHS} | "
          f"Train: {history['train_total_loss'][-1]:.6f} | "
          f"Val: {history['val_total_loss'][-1]:.6f} | "
          f"L_aux: {history['val_aux_loss'][-1]:.6f} | "
          f"L0: {history['val_l0_sparsity'][-1]:.1f} | "
          f"{width_str}")

    # Early stopping (basado en loss total de validación)
    if history['val_total_loss'][-1] < best_val_loss - 1e-4:
        best_val_loss     = history['val_total_loss'][-1]
        best_epoch        = epoch + 1
        epochs_no_improve = 0
        torch.save(model.state_dict(), BEST_CKPT)
        save_checkpoint(
            model,
            base_path=NAMING_META['base_path'],
            variante=NAMING_META['variante'],
            tecnica=NAMING_META['tecnica'],
            capa=NAMING_META['capa'],
            juegos=NAMING_META['juegos'],
            estado='best',
            extras=NAMING_META['extras'],
            config=cfg,
            history=history,
            optimizer=optimizer
        )
        print(f'  Mejor modelo guardado (época {best_epoch} | Val Loss: {best_val_loss:.6f})')

    else:
        epochs_no_improve += 1
        if epochs_no_improve >= PATIENCE:
            print(f'\nEarly stopping en época {epoch+1} (mejor: época {best_epoch})')
            break

# Restaurar mejor modelo
model.load_state_dict(torch.load(BEST_CKPT, weights_only=True))
print(f'\nMejor época: {best_epoch} | Mejor Val Loss: {best_val_loss:.6f}')
```

```
Iniciando entrenamiento: 1000 épocas máximo
Early stopping: patience=20
```

```
k = 50 features activas (L0 promedio  k por batch)
Anchos matryoshka: [2048, 4096, 8192]
Best checkpoint: saved_model\sae_matryoshkabatchtopk_matryoshkabatchtopk_16_12500g_ex2_best.p
======================================================================
Época   1/1000 | Train: 0.658809 | Val: 0.891210 | L_aux: 0.000000 | L0: 208.8 | w2048=0.5443
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 1 | Val Loss: 0.891210)
Época   2/1000 | Train: 0.552006 | Val: 0.520370 | L_aux: 0.000867 | L0: 57.2 | w2048=0.3971
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 2 | Val Loss: 0.520370)
Época   3/1000 | Train: 0.517424 | Val: 0.514295 | L_aux: 0.000000 | L0: 47.4 | w2048=0.3908
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 3 | Val Loss: 0.514295)
Época   4/1000 | Train: 0.499381 | Val: 0.502527 | L_aux: 0.000000 | L0: 47.2 | w2048=0.3756
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 4 | Val Loss: 0.502527)
Época   5/1000 | Train: 0.488661 | Val: 0.493276 | L_aux: 0.000000 | L0: 47.8 | w2048=0.3642
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 5 | Val Loss: 0.493276)
Época   6/1000 | Train: 0.480976 | Val: 0.485838 | L_aux: 0.000000 | L0: 48.6 | w2048=0.3548
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 6 | Val Loss: 0.485838)
Época   7/1000 | Train: 0.475229 | Val: 0.480706 | L_aux: 0.000000 | L0: 49.0 | w2048=0.3484
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 7 | Val Loss: 0.480706)
Época   8/1000 | Train: 0.470644 | Val: 0.476846 | L_aux: 0.000000 | L0: 49.2 | w2048=0.3436
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 8 | Val Loss: 0.476846)
Época   9/1000 | Train: 0.467036 | Val: 0.473749 | L_aux: 0.000000 | L0: 49.2 | w2048=0.3396
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 9 | Val Loss: 0.473749)
Época  10/1000 | Train: 0.464045 | Val: 0.470976 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3358
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 10 | Val Loss: 0.470976)
Época  11/1000 | Train: 0.461474 | Val: 0.469099 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3334
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 11 | Val Loss: 0.469099)
Época  12/1000 | Train: 0.459297 | Val: 0.466989 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3303
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 12 | Val Loss: 0.466989)
Época  13/1000 | Train: 0.457505 | Val: 0.465301 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3279
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 13 | Val Loss: 0.465301)
```

```
Época  14/1000 | Train: 0.455779 | Val: 0.463908 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3261
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 14 | Val Loss: 0.463908)
Época  15/1000 | Train: 0.454382 | Val: 0.462721 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3244
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 15 | Val Loss: 0.462721)
Época  16/1000 | Train: 0.452930 | Val: 0.461581 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3228
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 16 | Val Loss: 0.461581)
Época  17/1000 | Train: 0.451899 | Val: 0.460458 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3211
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 17 | Val Loss: 0.460458)
Época  18/1000 | Train: 0.450770 | Val: 0.459460 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3197
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 18 | Val Loss: 0.459460)
Época  19/1000 | Train: 0.449851 | Val: 0.458678 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3187
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 19 | Val Loss: 0.458678)
Época  20/1000 | Train: 0.448920 | Val: 0.457954 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3177
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 20 | Val Loss: 0.457954)
Época  21/1000 | Train: 0.448096 | Val: 0.457376 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3169
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 21 | Val Loss: 0.457376)
Época  22/1000 | Train: 0.447303 | Val: 0.456600 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3159
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 22 | Val Loss: 0.456600)
Época  23/1000 | Train: 0.446705 | Val: 0.455937 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3149
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 23 | Val Loss: 0.455937)
Época  24/1000 | Train: 0.446051 | Val: 0.455453 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3144
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 24 | Val Loss: 0.455453)
Época  25/1000 | Train: 0.445414 | Val: 0.454970 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3138
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 25 | Val Loss: 0.454970)
Época  26/1000 | Train: 0.444876 | Val: 0.454494 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3131
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 26 | Val Loss: 0.454494)
Época  27/1000 | Train: 0.444396 | Val: 0.454192 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3128
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 27 | Val Loss: 0.454192)
Época  28/1000 | Train: 0.443887 | Val: 0.453949 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3125
```

```
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 28 | Val Loss: 0.453949)
Época  29/1000 | Train: 0.443482 | Val: 0.453761 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3123
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 29 | Val Loss: 0.453761)
Época  30/1000 | Train: 0.443045 | Val: 0.453153 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3116
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 30 | Val Loss: 0.453153)
Época  31/1000 | Train: 0.442644 | Val: 0.453178 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3118
Época  32/1000 | Train: 0.442372 | Val: 0.452760 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3113
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 32 | Val Loss: 0.452760)
Época  33/1000 | Train: 0.442066 | Val: 0.452439 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3110
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 33 | Val Loss: 0.452439)
Época  34/1000 | Train: 0.441754 | Val: 0.452111 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3105
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 34 | Val Loss: 0.452111)
Época  35/1000 | Train: 0.441446 | Val: 0.452006 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3106
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 35 | Val Loss: 0.452006)
Época  36/1000 | Train: 0.441143 | Val: 0.451707 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3103
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 36 | Val Loss: 0.451707)
Época  37/1000 | Train: 0.440892 | Val: 0.451466 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3101
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 37 | Val Loss: 0.451466)
Época  38/1000 | Train: 0.440543 | Val: 0.451293 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3101
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 38 | Val Loss: 0.451293)
Época  39/1000 | Train: 0.440370 | Val: 0.451112 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3099
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 39 | Val Loss: 0.451112)
Época  40/1000 | Train: 0.440235 | Val: 0.450698 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3094
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 40 | Val Loss: 0.450698)
Época  41/1000 | Train: 0.439969 | Val: 0.450761 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3096
Época  42/1000 | Train: 0.439823 | Val: 0.450690 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3096
Época  43/1000 | Train: 0.439612 | Val: 0.450561 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3095
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 43 | Val Loss: 0.450561)
Época  44/1000 | Train: 0.441968 | Val: 0.458753 | L_aux: 0.008098 | L0: 49.4 | w2048=0.3099
Época  45/1000 | Train: 0.444887 | Val: 0.458655 | L_aux: 0.008087 | L0: 49.4 | w2048=0.3100
```

```
Época  46/1000 | Train: 0.440792 | Val: 0.450930 | L_aux: 0.000216 | L0: 49.2 | w2048=0.3103
Época  47/1000 | Train: 0.439060 | Val: 0.451537 | L_aux: 0.001532 | L0: 49.6 | w2048=0.3092
Época  48/1000 | Train: 0.439338 | Val: 0.449688 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3087
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 48 | Val Loss: 0.449688)
Época  49/1000 | Train: 0.438620 | Val: 0.449646 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3088
Época  50/1000 | Train: 0.438499 | Val: 0.449641 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3088
Época  51/1000 | Train: 0.438418 | Val: 0.449515 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3087
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 51 | Val Loss: 0.449515)
Época  52/1000 | Train: 0.438260 | Val: 0.449388 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3086
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 52 | Val Loss: 0.449388)
Época  53/1000 | Train: 0.438142 | Val: 0.449686 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3092
Época  54/1000 | Train: 0.438057 | Val: 0.449276 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3086
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 54 | Val Loss: 0.449276)
Época  55/1000 | Train: 0.437930 | Val: 0.449106 | L_aux: 0.000000 | L0: 49.8 | w2048=0.3084
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 55 | Val Loss: 0.449106)
Época  56/1000 | Train: 0.437746 | Val: 0.449222 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3087
Época  57/1000 | Train: 0.437738 | Val: 0.448977 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3083
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 57 | Val Loss: 0.448977)
Época  58/1000 | Train: 0.437590 | Val: 0.448952 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3084
Época  59/1000 | Train: 0.437527 | Val: 0.448872 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3082
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 59 | Val Loss: 0.448872)
Época  60/1000 | Train: 0.437448 | Val: 0.448768 | L_aux: 0.000000 | L0: 49.8 | w2048=0.3081
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 60 | Val Loss: 0.448768)
Época  61/1000 | Train: 0.437275 | Val: 0.449090 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3088
Época  62/1000 | Train: 0.437234 | Val: 0.449050 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3087
Época  63/1000 | Train: 0.437080 | Val: 0.449039 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3088
Época  64/1000 | Train: 0.437000 | Val: 0.448926 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3087
Época  65/1000 | Train: 0.436873 | Val: 0.448886 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3087
Época  66/1000 | Train: 0.436803 | Val: 0.448640 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3084
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
   Mejor modelo guardado (época 66 | Val Loss: 0.448640)
Época  67/1000 | Train: 0.436784 | Val: 0.448795 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3086
Época  68/1000 | Train: 0.436704 | Val: 0.448652 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3084
Época  69/1000 | Train: 0.436653 | Val: 0.448848 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3088
Época  70/1000 | Train: 0.436592 | Val: 0.448397 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3082
```

```
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 70 | Val Loss: 0.448397)
Época  71/1000 | Train: 0.436475 | Val: 0.448476 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3083
Época  72/1000 | Train: 0.436356 | Val: 0.448473 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3083
Época  73/1000 | Train: 0.436323 | Val: 0.448449 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3084
Época  74/1000 | Train: 0.436236 | Val: 0.448430 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3083
Época  75/1000 | Train: 0.436158 | Val: 0.448687 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3088
Época  76/1000 | Train: 0.436189 | Val: 0.448345 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3083
Época  77/1000 | Train: 0.436057 | Val: 0.451174 | L_aux: 0.003020 | L0: 49.7 | w2048=0.3080
Época  78/1000 | Train: 0.436458 | Val: 0.448281 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3083
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 78 | Val Loss: 0.448281)
Época  79/1000 | Train: 0.435977 | Val: 0.448067 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3079
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 79 | Val Loss: 0.448067)
Época  80/1000 | Train: 0.435872 | Val: 0.448378 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3085
Época  81/1000 | Train: 0.435796 | Val: 0.448121 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3082
Época  82/1000 | Train: 0.435695 | Val: 0.448142 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3082
Época  83/1000 | Train: 0.441289 | Val: 0.450741 | L_aux: 0.002799 | L0: 49.7 | w2048=0.3080
Época  84/1000 | Train: 0.435644 | Val: 0.450403 | L_aux: 0.002795 | L0: 50.0 | w2048=0.3075
Época  85/1000 | Train: 0.435993 | Val: 0.450499 | L_aux: 0.002797 | L0: 49.8 | w2048=0.3076
Época  86/1000 | Train: 0.441395 | Val: 0.451252 | L_aux: 0.003015 | L0: 49.4 | w2048=0.3085
Época  87/1000 | Train: 0.436745 | Val: 0.450602 | L_aux: 0.002584 | L0: 49.6 | w2048=0.3082
Época  88/1000 | Train: 0.436115 | Val: 0.450179 | L_aux: 0.002155 | L0: 49.6 | w2048=0.3081
Época  89/1000 | Train: 0.435599 | Val: 0.448816 | L_aux: 0.000868 | L0: 49.6 | w2048=0.3080
Época  90/1000 | Train: 0.435348 | Val: 0.447910 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3080
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 90 | Val Loss: 0.447910)
Época  91/1000 | Train: 0.435259 | Val: 0.447935 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3081
Época  92/1000 | Train: 0.435234 | Val: 0.447815 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3080
Época  93/1000 | Train: 0.435171 | Val: 0.447902 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3080
Época  94/1000 | Train: 0.435168 | Val: 0.447546 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3075
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 94 | Val Loss: 0.447546)
Época  95/1000 | Train: 0.435127 | Val: 0.447843 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3081
Época  96/1000 | Train: 0.435037 | Val: 0.447529 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3075
Época  97/1000 | Train: 0.435040 | Val: 0.447380 | L_aux: 0.000000 | L0: 49.9 | w2048=0.3073
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 97 | Val Loss: 0.447380)
Época  98/1000 | Train: 0.435007 | Val: 0.447604 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3077
Época  99/1000 | Train: 0.434860 | Val: 0.448008 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3084
Época 100/1000 | Train: 0.434865 | Val: 0.447794 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3081
Época 101/1000 | Train: 0.434813 | Val: 0.447750 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3080
```

```
Época 102/1000 | Train: 0.434847 | Val: 0.447427 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3075
Época 103/1000 | Train: 0.434781 | Val: 0.447752 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3080
Época 104/1000 | Train: 0.434794 | Val: 0.447285 | L_aux: 0.000000 | L0: 49.8 | w2048=0.3073
Época 105/1000 | Train: 0.434655 | Val: 0.447687 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3080
Época 106/1000 | Train: 0.434638 | Val: 0.447471 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3076
Época 107/1000 | Train: 0.434651 | Val: 0.447327 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3074
Época 108/1000 | Train: 0.434583 | Val: 0.447383 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3075
Época 109/1000 | Train: 0.434556 | Val: 0.447631 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3078
Época 110/1000 | Train: 0.434523 | Val: 0.447633 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3079
Época 111/1000 | Train: 0.434496 | Val: 0.447461 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3076
Época 112/1000 | Train: 0.434413 | Val: 0.447655 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3079
Época 113/1000 | Train: 0.434470 | Val: 0.447306 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3074
Época 114/1000 | Train: 0.434322 | Val: 0.447521 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3077
Época 115/1000 | Train: 0.434328 | Val: 0.447362 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3075
Época 116/1000 | Train: 0.434321 | Val: 0.447087 | L_aux: 0.000000 | L0: 49.8 | w2048=0.3071
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 116 | Val Loss: 0.447087)
Época 117/1000 | Train: 0.434267 | Val: 0.447627 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3079
Época 118/1000 | Train: 0.434221 | Val: 0.447649 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3080
Época 119/1000 | Train: 0.434215 | Val: 0.447571 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3079
Época 120/1000 | Train: 0.434216 | Val: 0.447492 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3078
Época 121/1000 | Train: 0.434162 | Val: 0.447328 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3075
Época 122/1000 | Train: 0.434137 | Val: 0.447074 | L_aux: 0.000000 | L0: 49.8 | w2048=0.3072
Época 123/1000 | Train: 0.434064 | Val: 0.446932 | L_aux: 0.000000 | L0: 49.8 | w2048=0.3069
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 123 | Val Loss: 0.446932)
Época 124/1000 | Train: 0.434097 | Val: 0.447113 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3072
Época 125/1000 | Train: 0.434052 | Val: 0.447211 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3074
Época 126/1000 | Train: 0.434044 | Val: 0.447107 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3073
Época 127/1000 | Train: 0.433986 | Val: 0.446997 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3071
Época 128/1000 | Train: 0.433942 | Val: 0.447019 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3071
Época 129/1000 | Train: 0.433907 | Val: 0.447349 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3076
Época 130/1000 | Train: 0.433899 | Val: 0.447528 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3079
Época 131/1000 | Train: 0.433943 | Val: 0.446742 | L_aux: 0.000000 | L0: 49.9 | w2048=0.3067
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 131 | Val Loss: 0.446742)
Época 132/1000 | Train: 0.433829 | Val: 0.446950 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3070
Época 133/1000 | Train: 0.433784 | Val: 0.447462 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3079
Época 134/1000 | Train: 0.433755 | Val: 0.447232 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3075
Época 135/1000 | Train: 0.439572 | Val: 0.455492 | L_aux: 0.007902 | L0: 49.2 | w2048=0.3082
Época 136/1000 | Train: 0.437558 | Val: 0.446442 | L_aux: 0.000000 | L0: 50.1 | w2048=0.3065
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma
  Mejor modelo guardado (época 136 | Val Loss: 0.446442)
```

```
Época 137/1000 | Train: 0.434625 | Val: 0.447515 | L_aux: 0.000429 | L0: 49.6 | w2048=0.3074
Época 138/1000 | Train: 0.433674 | Val: 0.447399 | L_aux: 0.000000 | L0: 49.4 | w2048=0.3079
Época 139/1000 | Train: 0.433658 | Val: 0.447167 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3074
Época 140/1000 | Train: 0.433696 | Val: 0.447071 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3073
Época 141/1000 | Train: 0.433602 | Val: 0.447343 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3077
Época 142/1000 | Train: 0.433641 | Val: 0.446823 | L_aux: 0.000000 | L0: 49.8 | w2048=0.3069
Época 143/1000 | Train: 0.433620 | Val: 0.446905 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3071
Época 144/1000 | Train: 0.433608 | Val: 0.447071 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3073
Época 145/1000 | Train: 0.433548 | Val: 0.447309 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3077
Época 146/1000 | Train: 0.433528 | Val: 0.446886 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3070
Época 147/1000 | Train: 0.433490 | Val: 0.446951 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3071
Época 148/1000 | Train: 0.433417 | Val: 0.447059 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3073
Época 149/1000 | Train: 0.433451 | Val: 0.447001 | L_aux: 0.000000 | L0: 49.5 | w2048=0.3072
Época 150/1000 | Train: 0.433404 | Val: 0.446685 | L_aux: 0.000000 | L0: 49.8 | w2048=0.3067
Época 151/1000 | Train: 0.433411 | Val: 0.447008 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3072
Época 152/1000 | Train: 0.433405 | Val: 0.446817 | L_aux: 0.000000 | L0: 49.7 | w2048=0.3069
Época 153/1000 | Train: 0.433364 | Val: 0.446649 | L_aux: 0.000000 | L0: 49.8 | w2048=0.3067
Época 154/1000 | Train: 0.433322 | Val: 0.447222 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3075
Época 155/1000 | Train: 0.433316 | Val: 0.446985 | L_aux: 0.000000 | L0: 49.6 | w2048=0.3072
Época 156/1000 | Train: 0.433244 | Val: 0.447211 | L_aux: 0.000000 | L0: 49.3 | w2048=0.3076

Early stopping en época 156 (mejor: época 136)

Mejor época: 136 | Mejor Val Loss: 0.446442
```

8. Guardar Modelo

```python
# Guardar checkpoint final
checkpoint = {
    'model_state_dict':     model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'config': {
        'architecture':      cfg['architecture'],
        'd_in':              cfg['d_in'],
        'd_sae':             cfg['d_sae'],
        'k':                 cfg['k'],
        'matryoshka_widths': cfg['matryoshka_widths'],
        'lr':                cfg['lr'],
        'hook_name':         cfg['hook_name'],
    },
    'history': history,
```

```
}

final_model_name = get_model_name(
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='final',
    extras_id=extras_id
)
checkpoint_path = save_dir / final_model_name
torch.save(checkpoint, checkpoint_path)
print(f'Modelo guardado: {checkpoint_path}')

save_checkpoint(
    model,
    base_path=NAMING_META['base_path'],
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='final',
    extras=NAMING_META['extras'],
    config=cfg,
    history=history,
    optimizer=optimizer
)
```

```
Modelo guardado: saved_model\sae_matryoshkabatchtopk_matryoshkabatchtopk_16_12500g_ex2_final
  Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_ma

WindowsPath('C:/Users/Esposa/Documents/Repos/sae-othello-gpt/layer_06/models/sae_matryoshkaba
```

9. Métricas básicas del modelo

```
metrics_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
```

```
    NAMING_META['juegos'],
    extras_id=extras_id
)

training_metrics = {
    # Calidad de reconstrucción (loss total = media de MSE en todos los anchos)
    'val_total_loss_final':      history['val_total_loss'][-1],
    'val_total_loss_best':       best_val_loss,
    # MSE por ancho matryoshka (último epoch)
    **{f'val_mse_w{w}_final': history[f'val_mse_w{w}'][-1] for w in WIDTHS},
    # Sparsity (L0 del ancho máximo)
    'val_l0_final':              history['val_l0_sparsity'][-1],
    'val_fraction_active_final': history['val_fraction_active'][-1],
    # Contexto del entrenamiento
    'best_epoch':                best_epoch,
    'total_epochs':              len(history['train_total_loss']),
    'train_total_loss_final':    history['train_total_loss'][-1],
    'train_l0_final':            history['train_l0_sparsity'][-1],
    # Hiperparámetros MatryoshkaBatchTopK
    'architecture':              cfg['architecture'],
    'k':                         cfg['k'],
    'matryoshka_widths':         cfg['matryoshka_widths'],
    'd_sae':                     cfg['d_sae'],
    'd_in':                      cfg['d_in'],
    'expansion_factor':          cfg['d_sae'] // cfg['d_in'],
    'hook_name':                 cfg['hook_name'],
}

os.makedirs(metrics_dir, exist_ok=True)
metrics_path = Path(metrics_dir) / 'training_metrics.json'
with open(metrics_path, 'w') as f:
    json.dump(training_metrics, f, indent=2)

print(f'Métricas guardadas: {metrics_path}')
print(pd.DataFrame(
    list(training_metrics.items()), columns=['Métrica', 'Valor']
).to_string(index=False))
```

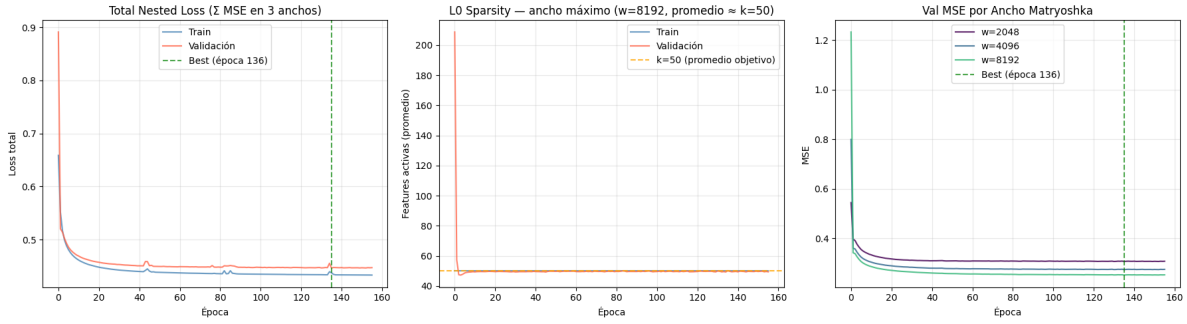
```
Métricas guardadas: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sa
                Métrica                  Valor
     val_total_loss_final               0.447211
      val_total_loss_best               0.446442
```

```
      val_mse_w2048_final                 0.307564
      val_mse_w4096_final                 0.274746
      val_mse_w8192_final                 0.252659
             val_l0_final                49.346077
val_fraction_active_final                 0.006024
               best_epoch                      136
             total_epochs                      156
   train_total_loss_final                 0.433244
            train_l0_final                     50.0
              architecture      matryoshkabatchtopk
                         k                       50
        matryoshka_widths       [2048, 4096, 8192]
                    d_sae                     8192
                     d_in                      512
          expansion_factor                       16
                 hook_name blocks.5.hook_resid_post
```

10. Visualización de Resultados

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Total Nested Loss
axes[0].plot(history['train_total_loss'], label='Train', color='steelblue', alpha=0.8)
axes[0].plot(history['val_total_loss'],   label='Validación', color='tomato', alpha=0.8)
if best_epoch > 0:
    axes[0].axvline(best_epoch - 1, color='green', linestyle='--', alpha=0.7,
                    label=f'Best (época {best_epoch})')
axes[0].set_title(f'Total Nested Loss (Σ MSE en {len(WIDTHS)} anchos)')
axes[0].set_xlabel('Época')
axes[0].set_ylabel('Loss total')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# L0 sparsity (ancho máximo, L0 promedio  k)
axes[1].plot(history['train_l0_sparsity'], label='Train', color='steelblue', alpha=0.8)
axes[1].plot(history['val_l0_sparsity'],   label='Validación', color='tomato', alpha=0.8)
axes[1].axhline(cfg["k"], color='orange', linestyle='--', alpha=0.8,
                label=f'k={cfg["k"]} (promedio objetivo)')
axes[1].set_title(f'L0 Sparsity - ancho máximo (w={cfg["d_sae"]}, promedio  k={cfg["k"]})')
axes[1].set_xlabel('Época')
axes[1].set_ylabel('Features activas (promedio)')
```

```
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# MSE por ancho matryoshka (validación)
colors_w = plt.cm.viridis([i / len(WIDTHS) for i in range(len(WIDTHS))])
for w, color in zip(WIDTHS, colors_w):
    axes[2].plot(history[f'val_mse_w{w}'], label=f'w={w}', color=color, alpha=0.85)
if best_epoch > 0:
    axes[2].axvline(best_epoch - 1, color='green', linestyle='--', alpha=0.7,
                    label=f'Best (época {best_epoch})')
axes[2].set_title('Val MSE por Ancho Matryoshka')
axes[2].set_xlabel('Época')
axes[2].set_ylabel('MSE')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(plots_dir / 'training_curves.png', dpi=300, bbox_inches='tight')
plt.show()

print('\n' + '='*50)
print('Gráficas guardadas exitosamente')
print('='*50)
```




```
==================================================
Gráficas guardadas exitosamente
==================================================
```

12. Generar PDF con Quarto

Una vez ejecutado todo el notebook, ejecutar el siguiente comando en la terminal para generar el PDF con Quarto:

```
exp_dir = get_experiment_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

pdf_name = get_report_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'training',
    extras_id=extras_id
)

notebook_name = 'train_sae_matryoska-batch-topk.ipynb'
output_path = exp_dir / pdf_name

print(f'Generando PDF con Quarto...')
print(f'Archivo de salida: {output_path}')

# Quarto no acepta rutas en --output, solo nombre de archivo
# Se usa --output-dir para el directorio y --output solo para el nombre
result = subprocess.run(
    f'quarto render {notebook_name} --to pdf --output-dir "{exp_dir}" --output {pdf_name}',
    shell=True,
    capture_output=True,
    text=True
)

if result.returncode == 0:
    print(f'PDF generado exitosamente: {output_path}')
else:
    print(f'Error al generar PDF:')
    print(result.stderr)
```

```
Generando PDF con Quarto...
Archivo de salida: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sa
PDF generado exitosamente: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\lay
```